

astrid-runtime/astrid

Astrid : un runtime de sécurité Rust très outillé en CI, mais 42 bugs ouverts, un seul mainteneur et des tests natifs difficiles à rejouer localement

Astrid is a portable, capability-secure operating system for composable software.

10277 ★

137 forks

216 issues ouvertes

Apache-2.0

<https://github.com/astrid-runtime/astrid>

Que fait ce projet et quels sont ses parcours critiques ?

Astrid est « a portable, capability-secure operating system for composable software » ¹: un runtime user-space sur macOS et Linux qui exécute des composants WebAssembly (« capsules ») dans un sandbox Wasmtime, sans autorité ambiante, chaque accès fichier/réseau/processus étant contrôlé par un jeton de capacité signé ². Projet jeune (créé le 2026-02-15 ³), 10277 étoiles ⁴, 216 issues ouvertes ⁵, dernier push le 2026-09-09 ⁶, version courante v2026.9.0 ⁷. Parcours critiques à couvrir, tirés du README ²: 1. Cycle de vie du daemon: `astrid start / status / capsule list / stop`, avec retrait de l'état dans `astrid.volume` à l'arrêt et restauration au redémarrage. 2. Initialisation d'une distribution: `astrid init --distro <source> (repo, manifeste local, bundle signé --offline)`, écriture d'un `Distro.lock` épinglant chaque capsule par hash BLAKE3. 3. Cycle de vie des capsules à chaud: `capsule new / build / install / update / remove` sans redémarrage. 4. Isolation par principal: `agent create`, `caps show`, `quota set`, `pair-device issue --scope use-only`; un principal ne doit jamais lire l'espace d'un autre, et le système « fails closed ». 5. Montage du stockage durable: `storage mount / status / sync / unmount` via FSKit (macOS 26+) ou FUSE (Linux). Chaque parcours traverse les mécanismes de sécurité indépendants (sandbox WASM, manifeste allow-list, ACL IPC, jeton de capacité, gate d'approbation, sandbox OS, chaîne d'audit): ce sont eux qu'il faut tester en négatif ². Windows est testé en CI mais absent des archives de la release ².

Comment est-il testé aujourd'hui ?

`.cargo/` config Cargo locale

`.dockerignore` exclusions du build Docker

`.gemini/` config agent Gemini

`.gitattributes` attributs git

`.github/` 20 workflows GitHub Actions

`.gitignore` exclusions git

`.gitmodules` sous-modules git

`CHANGELOG.md` journal des versions, 410 Ko

`CONTRIBUTING.md` guide de contribution

`Cargo.lock` verrou des dépendances Rust

`Cargo.toml` manifeste du workspace Rust

`LICENSE-APACHE` licence Apache 2.0

`LICENSE-MIT` licence MIT

`README.md` présentation du projet

`SECURITY.md` politique de sécurité

`changes/` notes de changement en attente

`clippy.toml` config lint clippy

`container/` définitions de conteneur

`crates/` code principal : 33 crates Rust, 32 membres du workspace

docs/ documentation

e2e/ scénarios TOML et fixtures bout en bout

fuzz/ tests de fuzzing

native/ code natif macOS

release/ config de release (nightly.toml)

rust-toolchain.toml version Rust épinglée

rustfmt.toml config formatage rustfmt

scripts/ scripts build, CI, release

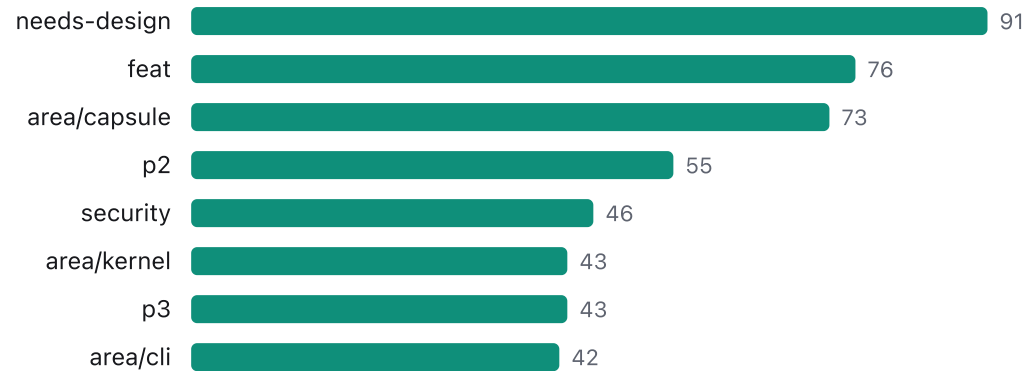
wit interfaces WIT, entrée de 0 octet

Pas de dossier `test/`, `tests/` ou `spec/` à la racine: les tests vivent sous `crates/` et `e2e/` ⁸. Le crate dédié est `crates/astrid-integration-tests` ⁹, 39 fichiers relevés ¹⁰, avec `cargo test` et des snapshots `insta` ¹¹; un crate `astrid-test` fournit `harness.rs` et `mocks.rs` ^{8 12}. Bout en bout: `e2e/` contient des scénarios TOML (`capability-scenarios.toml`, `cli-scenarios.toml`, `http-scenarios.toml`, `runtime-scenario-specs.toml`, `waiter-surfaces.toml`, `first-party-capsule-scenarios.toml`), un `concurrency.sh` et des fixtures de capsules adverses et concurrentes ^{13 8}. Un dossier `fuzz/` existe à la racine ¹⁴. Commande locale: `cargo test --workspace -- --quiet` ¹⁵; en CI, `cargo test --workspace --locked` sur ubuntu et `scripts/ci/test-workspace-macos.sh` sur macos ⁸ ¹⁶. Règle d'équipe: « Tests required. New features need tests. Bug fixes need a regression test » ¹⁷. Le cache ne contient ni taux de couverture, ni nombre de cas dans les scénarios TOML, ni le contenu de `runtime-e2e.yml` (« non lu »); à vérifier dans <https://github.com/astrid-runtime/astrid/tree/main/e2e> et <https://github.com/astrid-runtime/astrid/blob/main/.github/workflows/runtime-e2e.yml> ⁸.

Que dit la CI et quand tourne-t-elle ?

Un workflow principal, CI (`.github/workflows/ci.yml`), déclenché sur `push` et `pull_request` ^{18 19 20 21}. Le push ne déclenche que sur `main` et `stream-*` et seulement si des `.rs`, `Cargo.toml` / `Cargo.lock`, des scripts de release ou certains workflows changent; toute PR déclenche sans filtre (« Required build/test checks must also run for stacked and docs-only PRs ») ¹⁶. 14 jobs, toolchain Rust 1.95 épinglée, `harden-runner` sur chaque job ¹⁶: - Qualité: `fmt`, `clippy -D warnings`, `check --all-features`, `msrv` (1.95), `audit` (`cargo-audit` 0.22.2). - Tests: `test` (`ubuntu + macos`, `CARGO_PROFILE_TEST_DEBUG=0`, cf. #954), `linux-fuse-e2e` (un test `--ignored` de montage FUSE natif), `linux-v0104-upgrade` (script de migration depuis v0.10.4), `windows-filesystem-native` (WinFsp 2.1, x86_64 et aarch64), `wasm-portability`. - Release: `linux-release-smoke` (4 cibles `gnu/musl`, contrôles `glibc` ≤ 2.34 et ELF statique). - PR seulement, consultatifs: `api-compatibility` (`cargo-semver-checks`, cf. #1480), `public-rust-api-diff`, `gateway-openapi-contract`. Au total 20 workflows dans `.github/` ²², dont `runtime-e2e.yml`, `native-storage-certification.yml`, `supervised-storage-certification.yml`, `storage-benchmark.yml`, `windows-local-transport.yml`, `codeql.yml`, `nightly.yml` ¹⁶. Le risque CI est évalué « low » ^{23 24}. Le cache ne contient ni l'état des derniers runs ni les déclencheurs des 19 autres workflows (le MCP n'expose pas les runs Actions); à vérifier dans <https://github.com/astrid-runtime/astrid/actions>.

Où sont les bugs connus ?



192 issues ouvertes ²⁵, 149 fermées sur 30 jours ²⁶. Étiquettes utiles pour un plan de test: bug 42 ²⁷, security 46 ²⁸, test 16 ²⁹, blocked 16 ³⁰; priorités p1 36 ³¹, p2 55 ³², p3 43 ³³. Zones les plus chargées: area/capsule 73 ³⁴, area/kernel 43 ³⁵, area/cli 42 ³⁶, area/runtime 25 ³⁷, area/core 25 ³⁸. 91 issues sont needs-design ³⁹: comportement non figé, donc cas de test à écrire avec prudence. Bugs concrets et reproductibles parmi les « good first »: #459 « Markdown parser in TUI fails on numbered lists >= 10 » ⁴⁰ ⁴¹, #276 « MCP interceptor: propagate tool_result.is_error instead of treating it as success » ⁴² ⁴³, #461 « is_known_tag coverage table is 18 vs 19 and omits ToolCancelRequest » ⁴⁴ ⁴⁵, #476 sur l'enregistrement dupliqué dans CapsuleRegistry ⁴⁶. L'issue la plus discutée est un ticket de test: #1708 « test(container): qualify the signed amd64 Linux-backed Astrid host profile », 29 commentaires ⁴⁷ ⁴⁸ ⁴⁹. Le cache ne contient pas le croisement bug × area/* ni la liste des 42 bugs; à vérifier dans <https://github.com/astrid-runtime/astrid/issues?q=is%3Aopen+label%3Abug>.

Comment reproduire et signaler un bug ici ?

Procédure imposée par CONTRIBUTING.md ¹⁷: 1. Chercher une issue existante, puis utiliser le template « bug report ». 2. Reproduire avant d'ouvrir: version ou commit affecté, étapes exactes, comportement attendu et observé, preuves. Les audits en masse, findings spéculatifs et rapports générés par IA non vérifiés sont refusés (« AI output is a drafting aid, not evidence »). 3. Vulnérabilité: jamais d'issue publique, passer par GitHub Security Advisories ¹⁷; une politique SECURITY.md existe ⁵⁰. 4. Pour corriger soi-même: issue assignée d'abord (« Every PR must be linked to an issue. No exceptions »), régression obligatoire pour un bug fix, commit `git commit -s` (DCO), fragment `changes/{issue}.fixed.md`, titre Conventional Commits, template PR complet sinon rejet CI ¹⁷. Le flux est actif: 24 PR ouvertes ⁵¹ et une longue liste de correctifs fusionnés en 30 jours, presque tous en `fix(...)`, par exemple #1906 « fix(init): honor explicit grants after completed batch resume » ^{52 53} et #1870 « fix(test): provision private Windows run directory » ⁵⁴. Une PR de test récente donne le modèle d'une fixture négative: #1849 « test(capsule): use an unsatisfiable astrid-version fixture » ^{55 56}. Le cache ne contient pas le contenu des templates d'issue (`.github/ISSUE_TEMPLATE`); à vérifier dans <https://github.com/astrid-runtime/astrid/tree/main/.github>.

Quelles zones sont peu couvertes ?

Le cache ne contient aucune mesure de couverture de code; à vérifier dans <https://github.com/astrid-runtime/astrid/actions> (job `test`)⁸. Les signaux ci-dessous sont indirects. - Concentration humaine: bus factor 1⁵⁷, `joshuajbouw` 309 commits contre 7 pour le second contributeur^{58 59}; risque « high »^{60 61}. Le relecteur et l'auteur des tests sont la même personne: la revue QA externe a de la valeur. - Rythme soutenu sans creux: 41 commits en W26⁶², 17 en W37⁶³, dernier commit le 2026-09-09⁶⁴: les régressions arrivent vite, rejouer les scénarios `e2e/` à chaque release. - Plateformes: Windows testé en CI mais hors release²; le montage FUSE natif est un seul test `--ignored` en CI¹⁶; FSKit exige macOS 26+ et une approbation d'extension signée², difficile à automatiser. - Dépendances: 117 entrées dans `Cargo.toml`, 90 externes, 8 épinglées en exact^{65 66}; les deux PR Dependabot ouvertes (`#1834`, `#1877`)^{67 68} sont des candidats à une passe de non-régression. - Fonctionnel: 16 issues `test`²⁹ et 16 `blocked`³⁰; les thèmes chauds `campaign/os-universal` (32 issues⁶⁹) et les décisions en attente (`#1664` « specify the first-owner enrollment ceremony »⁷⁰, `#1692`⁷¹) n'ont pas encore de comportement spécifié à tester. - Le contenu de `fuzz/` et des 19 workflows hors `ci.yml` n'a pas été lu^{8 16}.

Plan de test en 5 points

1. Installer : `git clone https://github.com/astrid-runtime/astrid` `cd astrid` && `cargo build --release` # binary at `./target/release/astrid`
2. Lancer les tests : `cargo test --workspace -- --quiet`
3. Ouvrir `crates/astrid-cli/src/main.rs` — binaire ``astrid`` : `#[tokio::main]` qui parse clap (`cli::Cli`) puis route vers dispatch ; client léger qui se connecte au kernel (auto-démarrage), crée ou reprend des sessions et rend les événements en streaming. Confirmé (en-tête du fichier).

Sources

1. donnée collectée `repo.description`
2. source du repo `readme.md`
3. donnée collectée `repo.created_at`
4. donnée collectée `repo.stars`
5. donnée collectée `repo.open_issues`
6. donnée collectée `repo.pushed_at`
7. donnée collectée `releases.0.tag`
8. source du repo `tests.md`
9. donnée collectée `tests.dir`
10. donnée collectée `tests.files`
11. donnée collectée `tests.framework`
12. source du repo `tree.md`
13. donnée collectée `tree.20.role`
14. donnée collectée `tree.21.role`
15. donnée collectée `build.test`
16. source du repo `ci.md`
17. source du repo `contributing.md`
18. donnée collectée `build.ci.0.name`
19. donnée collectée `build.ci.0.path`

20. donnée collectée `build.ci.0.triggers.0`

21. donnée collectée `build.ci.0.triggers.1`

22. donnée collectée `tree.4.role`

23. donnée collectée `risks.3.level`

24. donnée collectée `risks.3.note`

25. donnée collectée `issues.open`

26. donnée collectée `issues.closed_30d`

27. donnée collectée `issues.by_label.bug`

28. donnée collectée `issues.by_label.security`

29. donnée collectée `issues.by_label.test`

30. donnée collectée `issues.by_label.blocked`

31. donnée collectée `issues.by_label.p1`

32. donnée collectée `issues.by_label.p2`

33. donnée collectée `issues.by_label.p3`

34. donnée collectée `issues.by_label.area/capsule`

35. donnée collectée `issues.by_label.area/kernel`

36. donnée collectée `issues.by_label.area/cli`

37. donnée collectée `issues.by_label.area/runtime`

38. donnée collectée `issues.by_label.area/core`

39. donnée collectée `issues.by_label.needs-design`

40. donnée collectée `issues.good_first.3.title`

41. donnée collectée `issues.good_first.3.url`

42. donnée collectée issues.good_first.4.title

43. donnée collectée issues.good_first.4.url

44. donnée collectée issues.good_first.2.title

45. donnée collectée issues.good_first.2.url

46. donnée collectée issues.good_first.1.title

47. donnée collectée issues.hot.0.title

48. donnée collectée issues.hot.0.comments

49. donnée collectée issues.hot.0.url

50. donnée collectée business.security_policy

51. donnée collectée pulls.open

52. donnée collectée pulls.merged_30d.0.title

53. donnée collectée pulls.merged_30d.0.url

54. donnée collectée pulls.merged_30d.17.title

55. donnée collectée pulls.merged_30d.28.title

56. donnée collectée pulls.merged_30d.28.url

57. donnée collectée activity.bus_factor

58. donnée collectée activity.contributors.0.commits

59. donnée collectée activity.contributors.1.commits

50. donnée collectée risks.1.level

51. donnée collectée risks.1.note

52. donnée collectée activity.commits_per_week.0.count

53. donnée collectée activity.commits_per_week.11.count

- 34. donnée collectée `activity.last_commit`
- 35. donnée collectée `deps.count`
- 36. donnée collectée `risks.2.note`
- 37. donnée collectée `roadmap.open_prs.0.title`
- 38. donnée collectée `roadmap.open_prs.1.title`
- 39. donnée collectée `issues.by_label.campaign/os-universal`
- 70. donnée collectée `issues.hot.6.title`
- 71. donnée collectée `issues.hot.8.title`